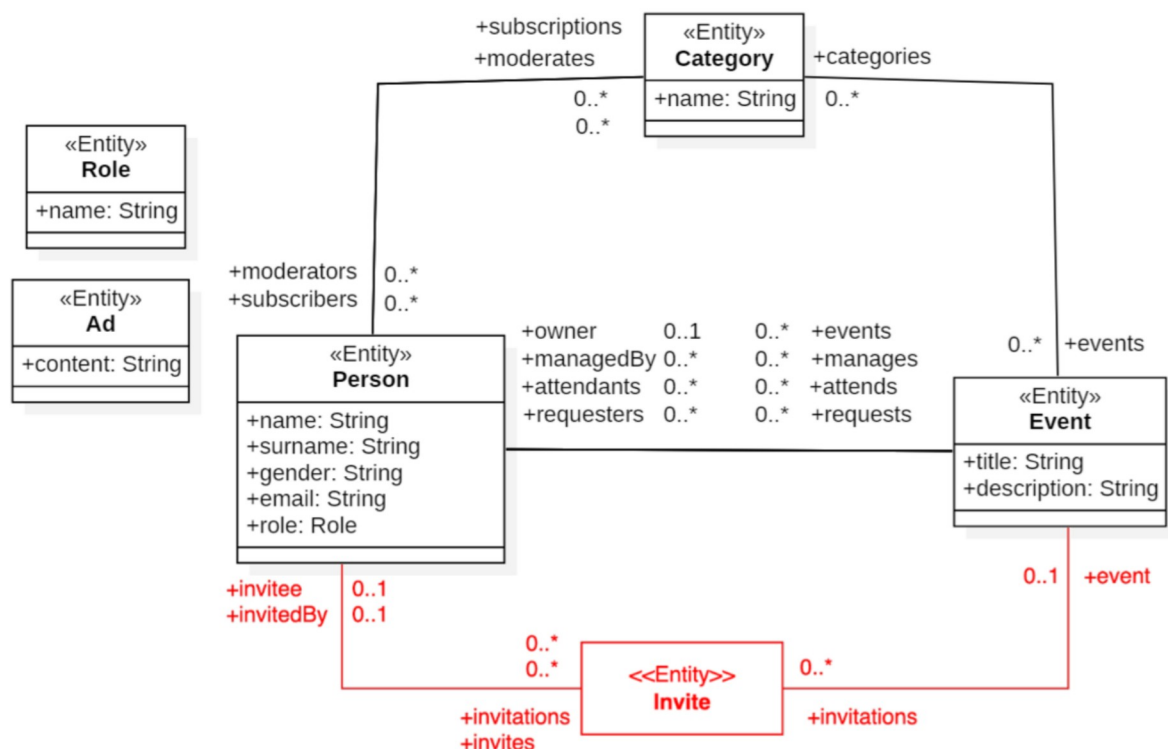


ChangeRequest

In the previous parts of the project you were tasked to implement a secure Event Platform web application twice, once using `νActionGUI` and once using `Flask`. As a starting point, this part assumes that the functional, security and privacy requirements described in the previous parts have been implemented in both `νActionGUI` and `Flask`. This part focuses on the changes to the web application to be implemented in the third part of the project.

1. Functional requirements



The Event Platform's data model must be extended to support invitations to events. An invitation is an entity that relates an event with two users: the one who created the invitation and the other who is invited to the event.

The above figure shows a class diagram representing the new extended data model. The addition of the **Invite** entity and the relevant association is marked in red. The appropriate association ends must be added to the existing entities as well as to the **Invite** entity. As an

example, (i) an event can contain zero or more invitations, these invitations are in the event's `invitations` association end; (ii) an invitation is sent from one user to another, which are values of the invitation's `invitedBy` and `invitee` association ends, respectively; (iii) an invitation belongs to one event, i.e., the event is the value of the invite's `event` association end. Finally, the **Invite** entity does not have any attributes.

Upon creation of an **Invite** entity object, its association ends are set in the following order: `event`, `invitedBy`, `invitee`. When writing security permissions in vActionGUI, you can assume this order. Invitations are read-only, i.e., once they are initially set, their association ends cannot be modified. It is only possible to delete the invitation.

The application is also extended to generalize the analytics functionality to insights that, in addition to administrators, provide to regular users personalized statistics about events they attend, manage, are invited to, and categories they subscribe to. Each user will have access to a page in their profile that summarizes such statistics.

We will update both of your projects to consistently change the data model and some endpoints functionality. More details are provided in the latter part of this section. To ensure that grading tests work properly, it is important that you do not change the data model or the other template files on your own.

2. Security requirements

We now focus on the extension of the authorization policy (i.e., the set of additional permissions) relevant for the invitations.

Stateful Permissions

Event

An event manager can read the list of invitations associated with the event they manage (the `invitations` association end) and read each **Invite** entity in the list.

Person

A user can read the lists of invitations they previously received or sent (the `invitations` and `invites` association ends, respectively) and read each **Invite** entity in the two lists.

Invite

To read an invitation (i.e., **Invite** entity) means that one can read all its association ends (i.e., `event`, `invitee`, and `invitedBy`).

Any authenticated user can create an invitation. Event managers can associate invitations to events they manage. They may assign themselves as the `invitedBy` for an invitation only if they manage the event associated with it. Additionally, they can invite users to events they manage by associating them as `invitees` of their invitation. Any user can cancel invitations that they have sent. Event managers can cancel any invitation sent for the events they manage. Invitation is canceled by being deleted (in this case, its association ends are never explicitly unset).

A user who has already requested to join an event or is already in the attendance list cannot be invited to it. Similarly, if a user has been invited to an event, they cannot request to join the event.

An invited user can accept or decline the invitation (in both cases, the invitation is deleted). Any user can add themselves as attendant to an event, if they have been invited to the event. In this case, the attendant list is updated before the invitation is deleted.

Invitations are read-only, i.e., once they have been initially assigned, the values of `invitee`, `invitedBy`, and `event` association ends are not allowed to be changed.

3. Privacy requirements

We now focus on additional data protection requirements.

Personal data

In the Event Platform, the following are considered additional personal data: the `attends` association end in the **Person** entity.

Purposes

For this application, we consider the following set of purposes:

- `Any`: This is the most general, top-level purpose. This purpose contains Marketing, ~~Analytics~~ Insights, and Functional purposes.
- `Insights`: Includes the existing Analytics purpose and a new Stats purpose.
 - `Analytics`: This focuses on analyzing users' data to generate insights and statistics. Specifically, an administrator can analyze users attending an event.
 - `Stats`: This provides personalized insights to the user on their profile page.

Remaining (complex and basic) purposes stay the same. Functions managing invitations are considered to implement functionality related to the `Core` purpose.

Privacy policy

A privacy policy (or notice) contains the following additional declared purposes:

- your `attends` for `Functional` Purpose.
- your `name`, `subscriptions` and `attends` for `Stats` Purpose, *if you attended more than 2 events.*

4. Template

The update template contains changes to the data model of both `vActionGUI` and `Flask` implementations consistent with what is presented in the above figure. In `Flask`, the update additionally extends the DTOs to accommodate the new **Invite** entity.

In both projects, we have added the options to view personal statistics (using the `personalized_stats()` endpoint) and see received invitations (in the `profile` template page). Users can also accept or decline invitations (using the `accept_invitation()` and `decline_invitation()` endpoints). In `Flask`, the `profile` template page traverses the additional invitations association end of the `PersonDTO` (rather than **Person** entity). Your implementation of the `profile` endpoint in `project.py` must be adjusted to support this.

In both `vActionGUI` and `Flask`, the corresponding functionality has been added to the `manage_event` template page (In particular, we include the list of all users that can be invited, i.e., invitees, as an input to the template). Your implementation of the `manage_event` endpoint in `project.py` of `Flask` must be adjusted to support this. Endpoints `send_invite`, `accept_invitation`, and `decline_invitation` have also been added to the data model (in `vActionGUI`) and their implementation to `project.py` (in both `vActionGUI` and `Flask`).

The `accept_invitation` (`decline_invitation`, respectively) endpoint takes an id of an invitation as a parameter and accepts (decline) it. The `send_invite` takes an id and e as parameters corresponding to a user and event ids, and it creates an invitation on behalf of the current user for the provided user and the event.

Note: Since the database schema is updated, you need to remove the previous database schema. In the case of `vActionGUI`, remember to regenerate the application (still using the `-re` tag).